

Análise de Algoritmos de Escalonamento

Analysis of Scheduling Algorithms

Cayo César¹, Gabriel José²

¹Universidade Federal do Rio Grande do Norte (UFRN)
– 59.300-000 – Caicó – RN – Brasil

²Departamento de Computação e Tecnologia
Universidade Federal do Rio Grande do Norte (UFRN) – Caicó, – RN Brasil

cayo.lobes.129@ufrn.edu, gabriel.aquino.069@ufrn.edu.br

Abstract. *This work presents the implementation and comparative analysis of process scheduling algorithms using the sched_sim simulator. The algorithms SJF (Shortest Job First), LJF (Longest Job First), PRIO STATIC, PRIO DYNAMIC, and PRIO DYNAMIC QUANTUM were developed based on the previously available FIFO algorithm. The main objective was to evaluate the impact of different scheduling strategies on operating system performance using the Average Waiting Time (AWT) as the primary metric. A total of 5400 simulations were performed by varying the number of processes and quantum values. The obtained results were statistically analyzed through the calculation of mean and standard deviation, and later presented in graphs with error bars. The analysis made it possible to compare the behavior of the algorithms under increasing system load and quantum changes, identifying the advantages, limitations, and performance characteristics of each scheduling approach.*

Keywords: *process scheduling, operating systems, average waiting time, scheduling algorithms, performance analysis.*

Resumo. *Este trabalho apresenta a implementação e a análise comparativa de algoritmos de escalonamento de processos utilizando o simulador sched_sim. Foram desenvolvidos os algoritmos SJF (Shortest Job First), LJF (Longest Job First), PRIO STATIC, PRIO DYNAMIC e PRIO DYNAMIC QUANTUM, tomando como base o algoritmo FIFO previamente disponibilizado. O objetivo principal consistiu em avaliar o impacto das diferentes estratégias de escalonamento no desempenho do sistema operacional, utilizando como métrica o Tempo Médio de Espera (TME). Para isso, foram realizadas 5400 simulações variando-se a quantidade de processos e os valores de quantum. Os resultados obtidos foram analisados estatisticamente por meio do cálculo da média e do desvio padrão, sendo posteriormente apresentados em gráficos com barras de erro. A análise permitiu comparar o comportamento dos algoritmos diante do aumento da carga do sistema e das mudanças no quantum, identificando vantagens, limitações e características de desempenho de cada abordagem de escalonamento.*

Palavras-chave: *escalonamento de processos, sistemas operacionais, tempo médio de espera, algoritmos de escalonamento, análise de desempenho.*

1. Introdução

O escalonador de processos desempenha um papel fundamental nos Sistemas Operacionais, sendo responsável por gerenciar o uso da CPU de maneira eficiente e justa entre os processos aptos à execução. Sua principal função consiste em selecionar qual processo utilizará o processador em determinado instante, buscando otimizar o desempenho total do sistema e garantir melhor aproveitamento dos recursos computacionais. Em ambientes multiprogramados, de tempo compartilhado (*time-sharing*) e de tempo real, diferentes políticas de escalonamento podem ser empregadas de acordo com os requisitos de responsividade e as características das cargas de trabalho.

Neste contexto, o presente trabalho tem como objetivo implementar e analisar diferentes algoritmos de escalonamento por meio de simulações computacionais. A avaliação busca compreender como cada estratégia influencia o desempenho do sistema, utilizando como principal métrica o Tempo Médio de Espera (TME). Para proporcionar uma análise mais abrangente, os experimentos consideraram diferentes cargas de trabalho, variando a quantidade de processos entre 10 e 100, além da utilização de diferentes valores de quantum (10, 20 e 30 μs) em cenários de escalonamento preemptivo.

O ambiente experimental foi construído utilizando o simulador acadêmico *sched.sim*, disponibilizado na disciplina DCT2101 – Sistemas Operacionais. A ferramenta, baseada em POSIX Threads, foi desenvolvida com finalidade educacional para auxiliar no estudo prático das estratégias de escalonamento. A versão inicial do simulador fornecia uma implementação funcional do algoritmo *First-In, First-Out* (FIFO), utilizada como base para o desenvolvimento dos demais algoritmos exigidos na atividade. A partir dessa estrutura, foram implementados os algoritmos SJF, LJF, PRIO_STATIC, PRIO_DYNAMIC e PRIO_DYNAMIC_QUANTUM. Dessa forma, o simulador serviu como plataforma para a execução das simulações e coleta dos resultados, permitindo a obtenção de médias e desvios padrões utilizados na análise comparativa entre as diferentes políticas de escalonamento.

2. Implementação dos Algoritmos

Nesta seção será apresentada as estratégias de escalonamento implementadas no simulador, bem como a lógica utilizada em cada estratégia de seleção de processos. As implementações foram desenvolvidas em linguagem C a partir da estrutura base fornecida pelo simulador *sched.sim*, abrangendo desde algoritmos clássicos de fila única até abordagens com múltiplas filas e prioridades dinâmicas.

2.1. Algoritmos de Fila Única (FIFO, SJF e LJF)

Os algoritmos de fila única, como o nome já diz, utilizam uma única fila de processos prontos, diferenciando-se apenas pela estratégia de seleção adotada.

O algoritmo *First-In, First-Out* (FIFO) realiza o escalonamento estritamente pela ordem de chegada dos processos ao sistema, sem considerar tempo de execução ou prioridade.

O algoritmo *Shortest Job First* (SJF) seleciona o processo com menor tempo restante de execução, buscando minimizar o Tempo Médio de Espera (TME). Em cenários ideais, esta estratégia tende a produzir melhores resultados médios de espera.

Já o algoritmo *Longest Job First* (LJF) opera de forma inversa ao SJF, priorizando processos com maior tempo de execução. Essa abordagem tende a aumentar o tempo de espera dos processos menores, podendo ocasionar o chamado *Convoy Effect*, no qual processos curtos permanecem aguardando por longos períodos.

2.2. Escalonador PRIO STATIC

O algoritmo de prioridade estática utiliza duas filas de execução, realizando a classificação dos processos com base em seu tempo total estimado de processamento. A estratégia adotada considera o valor de `process_time_total` em relação ao limite máximo definido pela variável global `MAX_TIME`.

Processos cujo tempo total excede 30% de `MAX_TIME` são direcionados para a fila de menor prioridade, enquanto processos menores permanecem na fila de maior prioridade. A seleção entre as filas ocorre de forma probabilística, atribuindo 70% de chance para a fila principal e 30% para a fila secundária.

Além disso, foi implementado um mecanismo de *fallback*, garantindo que, caso a fila sorteada esteja vazia, o escalonador selecione processos disponíveis na fila adjacente, evitando ociosidade da CPU.

```
int chance = rand() % 100;

if (chance < 70) {
    if (!isempty(ready))
        selected = dequeue(ready);
    else
        selected = dequeue(ready2);
}
```

2.3. Escalonador PRIO DYNAMIC

Na abordagem de prioridade dinâmica, a fila atribuída ao processo pode ser alterada durante sua execução, de acordo com o comportamento observado enquanto utilizava a CPU.

Quando o processo sofre preempção e retorna ao estado `READY`, ele é reposicionado na fila principal. Em contrapartida, processos que deixam a CPU para realizar operações de Entrada/Saída, entrando no estado `BLOCKED`, são direcionados para a fila secundária após retornarem da operação de E/S.

Essa estratégia busca favorecer processos interativos, que normalmente realizam operações de E/S com maior frequência e utilizam menos tempo contínuo de processamento.

2.4. Escalonador PRIO DYNAMIC QUANTUM

O algoritmo de prioridade dinâmica baseado em consumo de quantum utiliza uma heurística voltada à distinção entre processos *CPU-bound* e *I/O-bound*. Para isso, o escalonador analisa quanto do quantum disponível foi efetivamente consumido pelo processo antes de sua saída da CPU.

Caso o processo utilize até 50% do quantum disponível, ele é tratado como um processo mais interativo e permanece na fila de maior prioridade. Caso ultrapasse esse

limite, o processo é classificado como mais intensivo em processamento e direcionado para a fila de menor prioridade.

```
int tempo_usado = current->process_time;
int metade = QUANTUM / 2;

if (tempo_usado <= metade) {
    current->queue = 0;
} else {
    current->queue = 1;
}
```

Essa abordagem permite adaptar dinamicamente o comportamento do escalonador conforme o perfil de execução dos processos ao longo das simulações.

3. Metodologia

Os experimentos foram estruturados a partir de um planejamento experimental padronizado, incluindo múltiplas execuções para cada configuração avaliada.

3.1. Ambiente de Desenvolvimento e Execução

A implementação dos algoritmos foi realizada de forma colaborativa em dois computadores utilizando o sistema operacional Windows. Entretanto, visando garantir compatibilidade com o simulador baseado em POSIX Threads e com o compilador GCC, a compilação e execução das simulações ocorreram em ambiente Linux por meio do *Windows Subsystem for Linux* (WSL).

Todo o fluxo de desenvolvimento, compilação e execução foi conduzido utilizando o terminal integrado do Visual Studio Code (VS Code), permitindo integração entre o ambiente de edição e o subsistema Linux utilizado nos experimentos.

3.2. Delineamento Experimental

Com o objetivo de reduzir impactos decorrentes da aleatoriedade presente na geração dos processos do simulador, foi definido pelo professor[Borges 2026]. nas instruções do trabalho uma matriz de testes abrangente, incluindo diferentes cargas de trabalho e configurações de quantum.

Os experimentos consideraram os seguintes parâmetros:

- Algoritmos avaliados: FIFO, SJF, LJF, PRIO_STATIC, PRIO_DYNAMIC e PRIO_DYNAMIC_QUANTUM;
- Quantidade de processos: variação linear entre 10 e 100 processos, com incrementos de 10;
- Valores de quantum: 10, 20 e 30 μs .

Para cada combinação de parâmetros foram executadas 30 rodadas independentes de simulação, permitindo a obtenção de uma amostragem estatística mais consistente.

Ao todo, o conjunto experimental foi composto por:

$$6 \times 10 \times 3 \times 30 = 5400$$

simulações, considerando todos os algoritmos, quantidades de processos, valores de quantum e repetições realizadas.

3.3. Automação e Processamento dos Dados

Devido ao elevado volume de execuções necessárias, o processo de coleta dos resultados foi automatizado por meio de um script em Bash (`run.sh`), responsável por compilar os algoritmos, executar as simulações e armazenar os valores do Tempo Médio de Espera (TME) em arquivos CSV. Posteriormente, os dados coletados foram processados utilizando a linguagem Python, com suporte das bibliotecas *Pandas* e *Matplotlib*. Essa etapa permitiu realizar os cálculos estatísticos de média e desvio padrão, além da geração dos gráficos utilizados na análise comparativa dos algoritmos de escalonamento.

3.4. Uso de Inteligência Artificial

Em conformidade com as diretrizes da atividade, declaramos que o uso de ferramentas de IA restringiu-se ao suporte na depuração de scripts de automação e na revisão textual (padrão SBC). A implementação da lógica dos algoritmos em C foi estritamente autoral, assegurando o cumprimento dos objetivos de aprendizagem da disciplina.

4. Análise de Desempenho e Resultados

A avaliação de desempenho dos algoritmos implementados foi realizada a partir da análise do Tempo Médio de Espera (TME) em diferentes cenários de carga do sistema. Para isso, observou-se o comportamento das políticas de escalonamento à medida que a quantidade de processos variou de 10 a 100, considerando também diferentes valores de quantum (10, 20 e 30 μs).

A comparação entre os algoritmos permitiu analisar características relacionadas à eficiência, estabilidade e escalabilidade das estratégias implementadas, evidenciando como diferentes critérios de escalonamento impactam diretamente o desempenho do sistema.

4.1. Análise do Cenário com Quantum = 10 μs

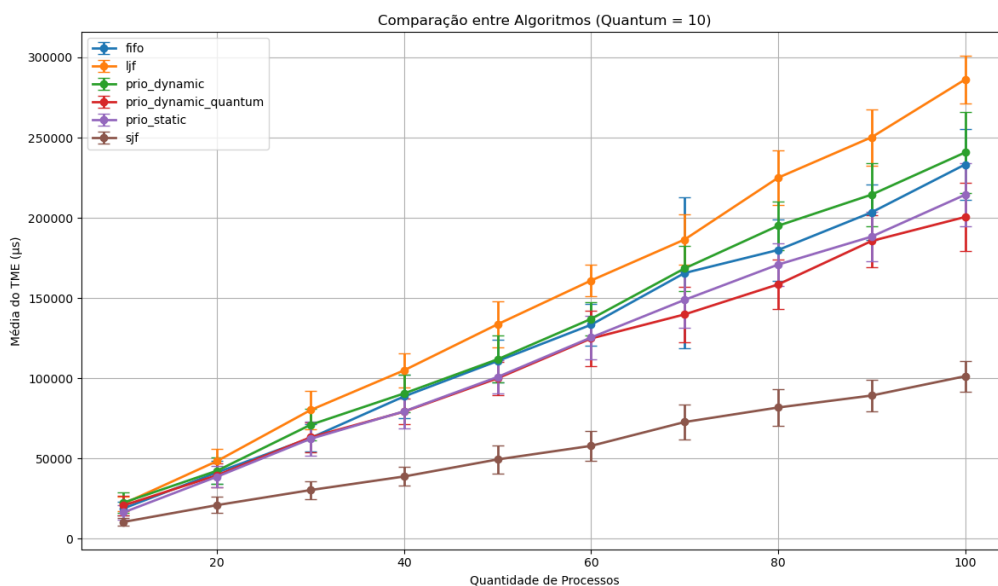


Figura 1. Comparação dos algoritmos com quantum de 10 μs .

A análise do primeiro cenário evidencia rapidamente os dois extremos clássicos das políticas de escalonamento. O algoritmo SJF apresentou os menores valores de TME de forma consistente ao longo de toda a simulação, demonstrando elevada eficiência na redução do tempo médio de espera. Esse comportamento ocorre devido à priorização de processos menores, reduzindo o acúmulo de tarefas na fila de prontos.

Em contrapartida, o algoritmo LJF apresentou os maiores valores de TME e um crescimento significativamente mais acentuado conforme o número de processos aumentou. Esse comportamento evidencia o chamado *Convoy Effect*, no qual processos menores permanecem aguardando por longos períodos devido à priorização constante de tarefas extensas.

Os algoritmos FIFO, PRIO_STATIC, PRIO_DYNAMIC e PRIO_DYNAMIC_QUANTUM apresentaram desempenho intermediário. Com o quantum reduzido para $10 \mu s$, observou-se aumento na quantidade de preempções e trocas de contexto, impactando diretamente o desempenho das estratégias dinâmicas.

4.2. Análise do Cenário com Quantum = $20 \mu s$

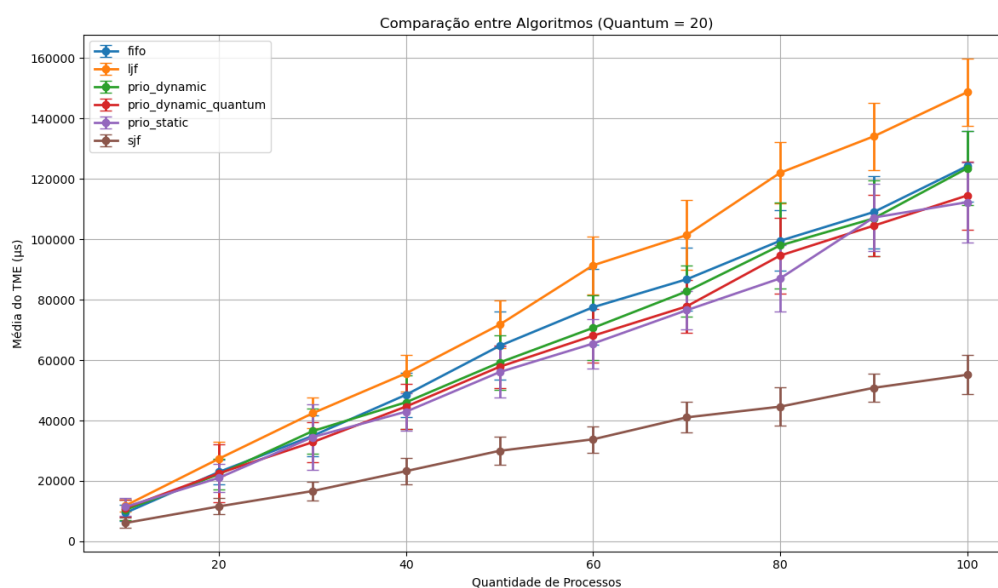


Figura 2. Comparação dos algoritmos com quantum de $20 \mu s$.

Com o aumento do quantum para $20 \mu s$, observou-se uma leve redução no crescimento das curvas de TME, indicando diminuição da sobrecarga provocada pelas trocas de contexto.

Nesse cenário, os algoritmos baseados em prioridade dinâmica demonstraram maior estabilidade e adaptação ao comportamento dos processos. O algoritmo PRIO_DYNAMIC_QUANTUM apresentou resultados mais equilibrados ao utilizar o consumo do quantum como critério para classificação dos processos entre as filas de prioridade.

A utilização de um quantum maior tornou mais precisa a distinção entre processos *CPU-bound* e *I/O-bound*, permitindo que os mecanismos de promoção e rebaixamento de

filas operassem de maneira mais eficiente.

4.3. Análise do Cenário com Quantum = 30 μs

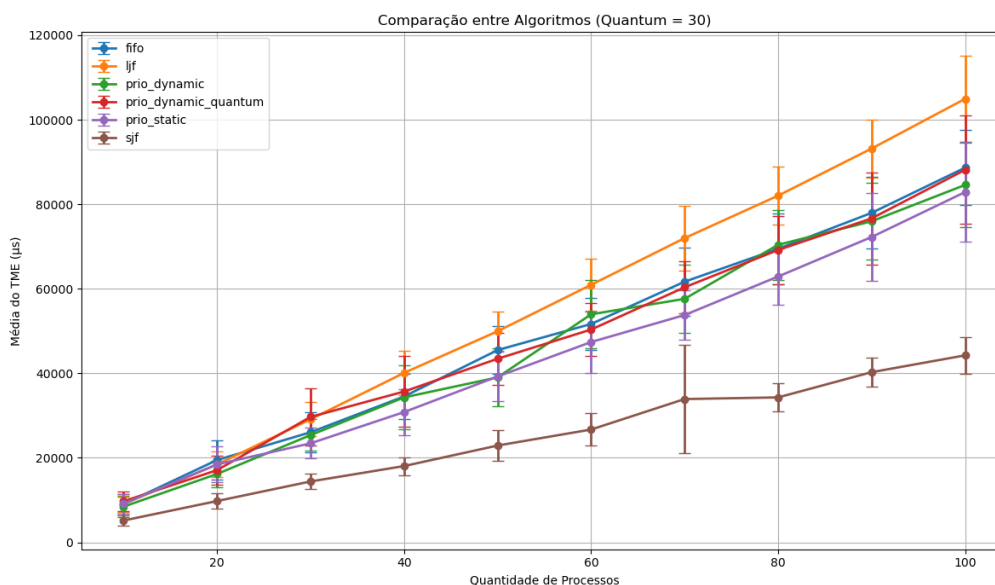


Figura 3. Comparação dos algoritmos com quantum de 10 μs .

No cenário com quantum de 30 μs , observou-se que os algoritmos preemptivos passaram a apresentar comportamento mais próximo de abordagens não preemptivas, especialmente para processos de curta duração, que frequentemente conseguiam finalizar sua execução dentro de um único quantum.

Independentemente do valor de quantum utilizado, todos os cenários evidenciaram o impacto direto do aumento da carga do sistema sobre o TME. À medida que o número de processos se aproximou de 100, verificou-se crescimento expressivo do tempo médio de espera em praticamente todos os algoritmos.

O algoritmo SJF manteve o comportamento mais estável e eficiente mesmo sob maior concorrência, apresentando crescimento mais suave em relação aos demais algoritmos. Já o LJF demonstrou baixa escalabilidade, com aumento acelerado do TME conforme a carga do sistema aumentava.

Os algoritmos de prioridade dinâmica apresentaram comportamento intermediário, evidenciando maior adaptabilidade em cenários com processos heterogêneos. Dessa forma, os resultados demonstram que a estratégia de escalonamento adotada exerce influência direta sobre o desempenho global do sistema, principalmente em ambientes com elevada concorrência por CPU.

5. Conclusão

Os resultados obtidos demonstraram comportamentos compatíveis com os conceitos clássicos de escalonamento de processos estudados em Sistemas Operacionais. O algoritmo SJF apresentou os menores valores de TME ao longo das simulações, confirmando

sua eficiência na minimização do tempo médio de espera, além de manter maior estabilidade mesmo em cenários com elevada quantidade de processos.

Por outro lado, o algoritmo LJF apresentou os maiores valores de TME e baixa escalabilidade, evidenciando na prática os impactos do chamado *Convoy Effect*, no qual processos menores permanecem aguardando por longos períodos devido à priorização de tarefas extensas.

Os algoritmos baseados em múltiplas filas e prioridades dinâmicas apresentaram resultados intermediários, demonstrando maior capacidade de adaptação em cenários heterogêneos. Em especial, o algoritmo PRIO_DYNAMIC_QUANTUM mostrou que a utilização de heurísticas simples, como a classificação de processos a partir do consumo do quantum, pode contribuir para um equilíbrio mais eficiente entre responsividade e utilização da CPU.

Além da análise dos algoritmos, o desenvolvimento deste trabalho reforçou a importância da simulação computacional e da análise estatística na avaliação de desempenho de sistemas operacionais. A utilização de múltiplas execuções, associada ao cálculo de médias e desvios padrões, permitiu obter resultados mais confiáveis e representativos, contribuindo para uma compreensão prática do impacto das diferentes políticas de escalonamento sobre o desempenho do sistema.

Referências

Borges, J. (2026). Atividade 2.1: Gerenciamento de processos – escalonamento.